SIMATS ENGINEERING

ASSESSMENT TOOL C02- AT 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.	BL5 – Evaluate

	Bloom's Level: BL6 – Create	
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze
8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

I _____MONIKA.B (192571002)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: MONIKA.B

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process

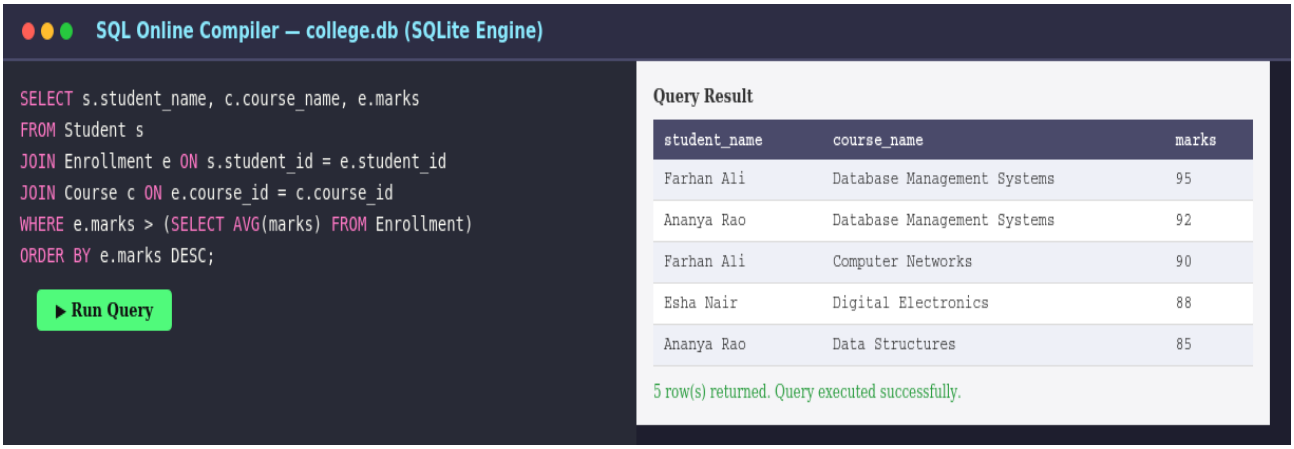
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

NAME: MONIKA.B
REG. NO: 192571002

Q1. **Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.**

```
SELECT s.student_name, c.course_name, e.marks
FROM Student s
JOIN Enrollment e ON s.student_id = e.student_id
JOIN Course c ON e.course_id = c.course_id
WHERE e.marks > (SELECT AVG(marks) FROM Enrollment)
ORDER BY e.marks DESC;
```

Implementation & Output:



Q2.Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.

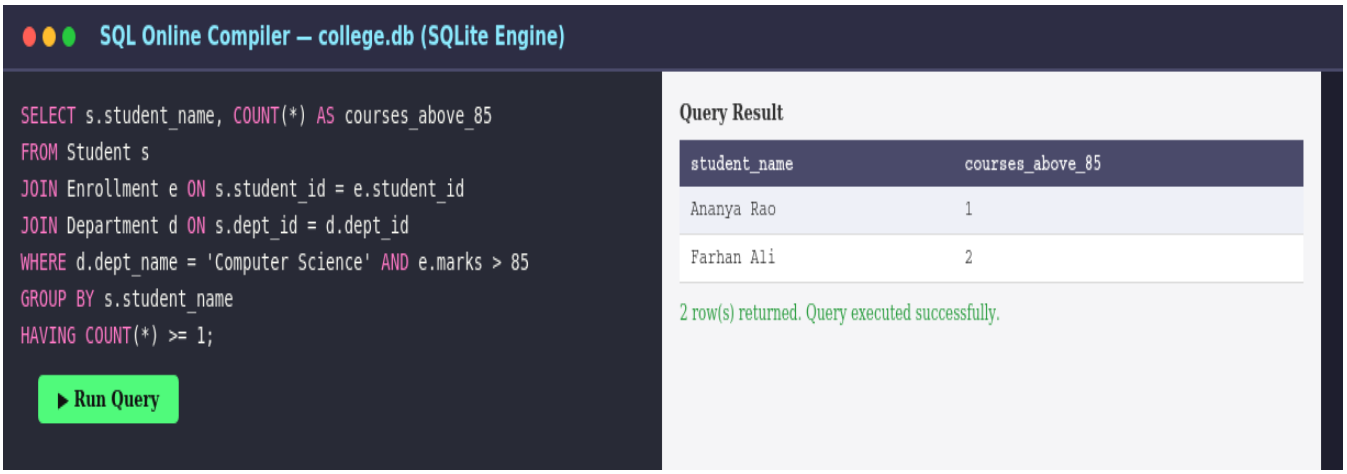
CODE:

```
CS_Students   ← σ(dept_name = 'Computer Science') (Department ⋈
Student)

HighScorers   ← σ(marks > 85) (Enrollment)

Qualified     ← CS_Students ⋈ HighScorers        (join on student_id)

Result        ← student_name γ COUNT(course_id) (Qualified)
```

Equivalent SQL implementation:

```
SELECT s.student_name, COUNT(*) AS courses_above_85
FROM Student s
JOIN Enrollment e ON s.student_id = e.student_id
JOIN Department d ON s.dept_id = d.dept_id
WHERE d.dept_name = 'Computer Science' AND e.marks > 85
GROUP BY s.student_name;
```

Implementation & Output:



Q3. Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.

CODE;

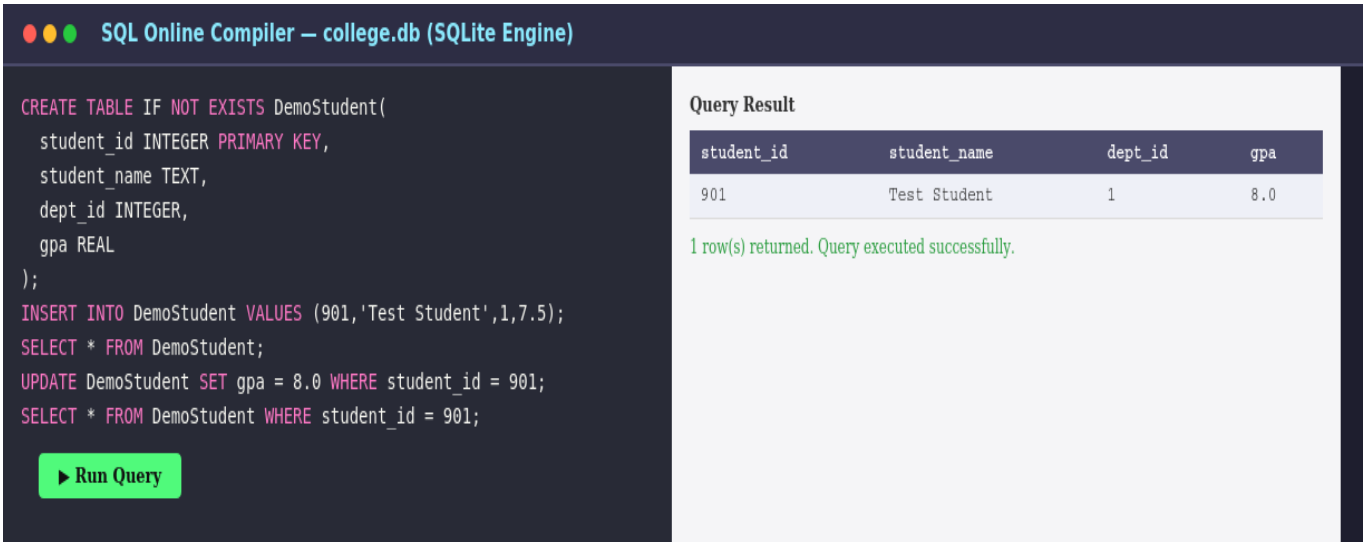
```
CREATE TABLE Department (
  dept_id INTEGER PRIMARY KEY,
  dept_name TEXT NOT NULL
);

CREATE TABLE Student (
  student_id INTEGER PRIMARY KEY,
  student_name TEXT NOT NULL,
  dept_id INTEGER REFERENCES Department(dept_id),
  gpa REAL
);

CREATE TABLE Enrollment (
  enroll_id INTEGER PRIMARY KEY,
  student_id INTEGER REFERENCES Student(student_id),
  course_id INTEGER REFERENCES Course(course_id),
  semester TEXT,
  marks INTEGER
);
```

Sample retrieval and update operations on the schema:

```
INSERT INTO DemoStudent VALUES (901, 'Test Student', 1, 7.5);
SELECT * FROM DemoStudent;
```

```
UPDATE DemoStudent SET gpa = 8.0 WHERE student_id = 901;
SELECT * FROM DemoStudent WHERE student_id = 901;
```

Implementation & Output:



Q4. Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.

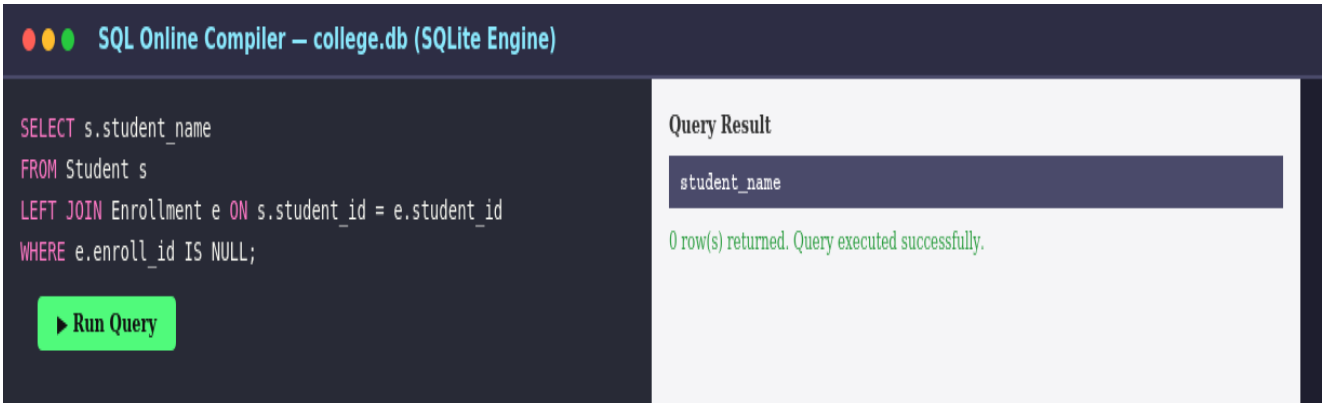
.

```
SELECT student_name
FROM Student
WHERE student_id IN (
    SELECT student_id
    FROM Enrollment
    WHERE marks > (
        SELECT AVG(marks) FROM Enrollment
    )
);
```

Implementation & Output:

Q5.Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency

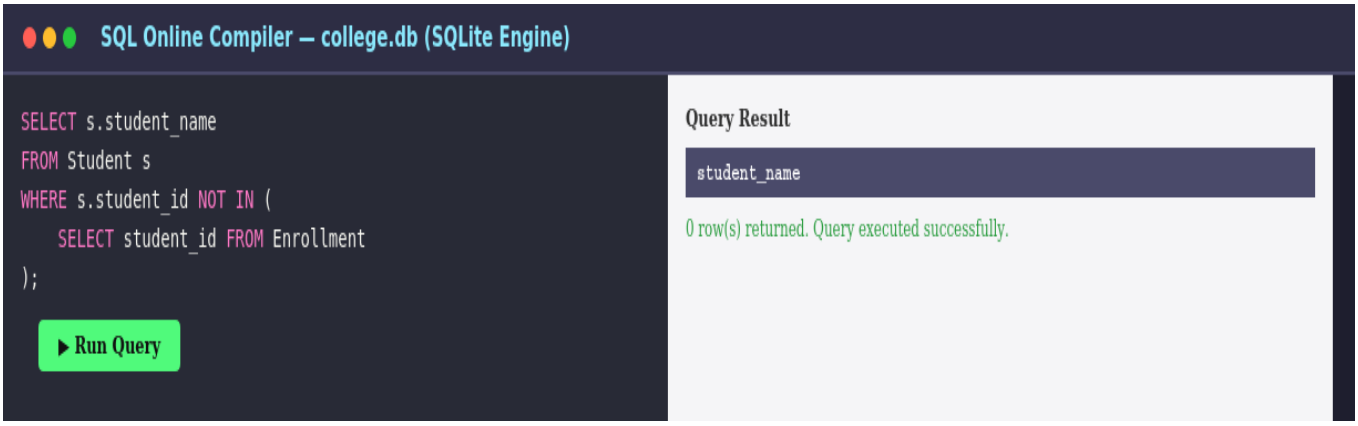
```
SELECT s.student_name
FROM Student s
LEFT JOIN Enrollment e ON s.student_id = e.student_id
WHERE e.enroll_id IS NULL;
```



Approach B — NOT IN subquery:

```
SELECT s.student_name
FROM Student s
WHERE s.student_id NOT IN (
```

```
    SELECT student_id FROM Enrollment
);
```



Q6.

Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.

.

Solution Process:

```
CREATE VIEW StudentPublicDirectory AS
SELECT s.student_name, d.dept_name
FROM Student s
JOIN Department d ON s.dept_id = d.dept_id;

SELECT * FROM StudentPublicDirectory;
```

Implementation & Output:

Q7.

Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

```
CS_Courses  ← σ(dept_name = 'Computer Science') (Course ⋈ Department)

Result      ← π(instructor_name) (Instructor ⋈ CS_Courses)
```

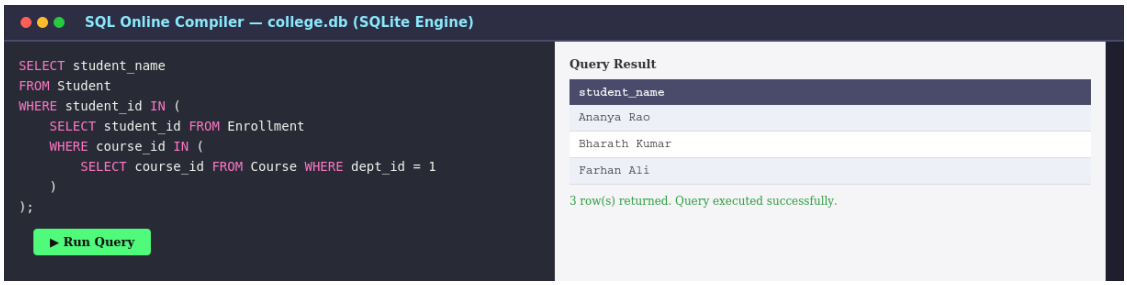
Equivalent SQL:

```
SELECT DISTINCT i.instructor_name
FROM Instructor i
JOIN Course c ON i.instructor_id = c.instructor_id
JOIN Department d ON c.dept_id = d.dept_id
WHERE d.dept_name = 'Computer Science';
```

Implementation & Output:

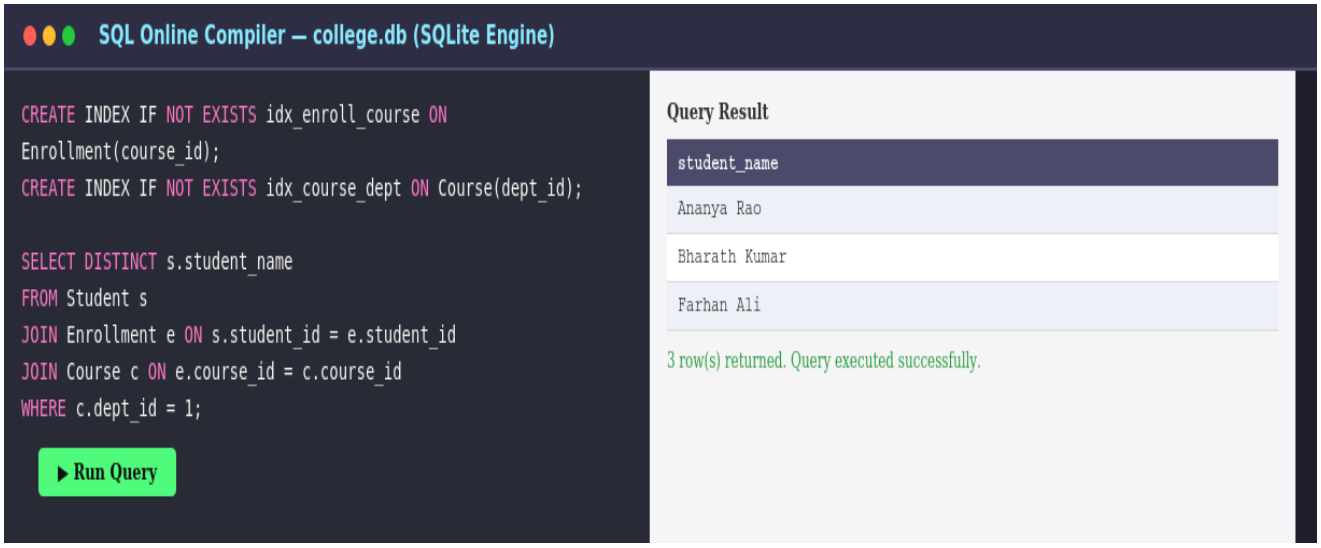
Q8. Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.

```
SELECT student_name
FROM Student
WHERE student_id IN (
    SELECT student_id FROM Enrollment
    WHERE course_id IN (
        SELECT course_id FROM Course WHERE dept_id = 1
    )
);
```



After — restructured with JOINs and supporting indexes:

```
CREATE INDEX idx_enroll_course ON Enrollment(course_id);
CREATE INDEX idx_course_dept ON Course(dept_id);

SELECT DISTINCT s.student_name
FROM Student s
JOIN Enrollment e ON s.student_id = e.student_id
JOIN Course c ON e.course_id = c.course_id
WHERE c.dept_id = 1;
```



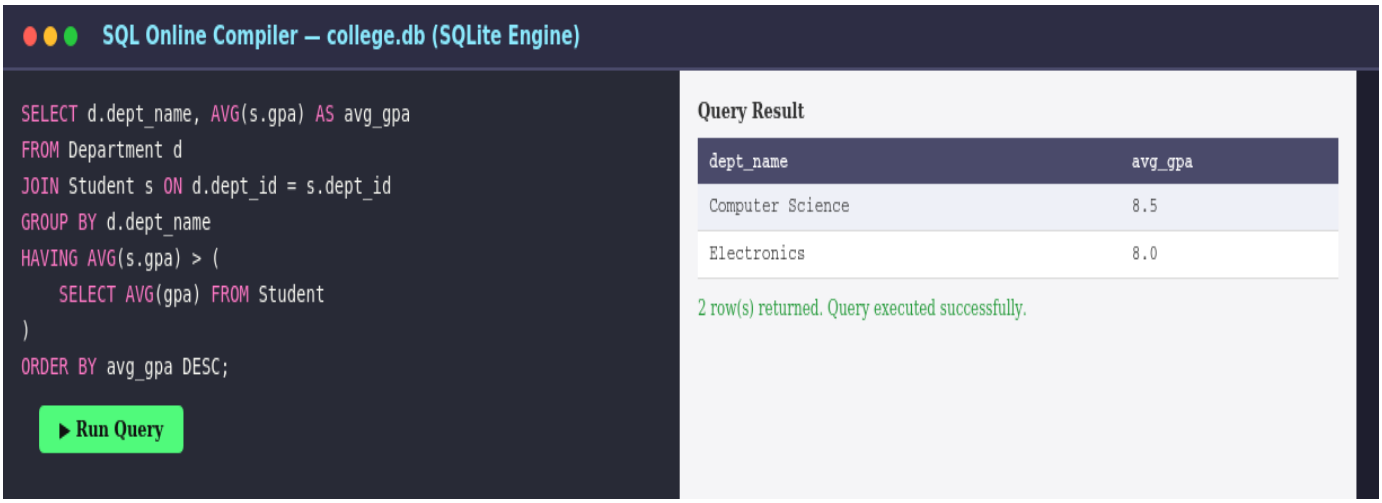
Q9.

Question: Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.

```
SELECT d.dept_name, AVG(s.gpa) AS avg_gpa
FROM Department d
JOIN Student s ON d.dept_id = s.dept_id
GROUP BY d.dept_name
HAVING AVG(s.gpa) > (
    SELECT AVG(gpa) FROM Student
```

```
)
ORDER BY avg_gpa DESC;
```

Implementation & Output:



Q10.

Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.

```
CREATE VIEW StudentCourseSummary AS
SELECT s.student_name, c.course_name, e.marks, d.dept_name
FROM Student s
JOIN Enrollment e ON s.student_id = e.student_id
JOIN Course c ON e.course_id = c.course_id
JOIN Department d ON s.dept_id = d.dept_id;

SELECT * FROM StudentCourseSummary WHERE marks >= 85;
```

Implementation & Output: